

MAGLALANG, SEAN GREI I.
C204

Finals Lab Task 5. CLI using Mysql and Python

1. Make sure you have installed the following pre-requisites before proceeding:
 - a. Mysql-connector
 - b. Mysql-connector-python
 - c. Xampp is running along with Apache and Mysql in the background
2. Create the following database in Mysql;
 - a. Database name: **moviesDB** with the ff: fields:

movie_id	int(10) Primary Key
title	varchar(50) NOT NULL
main_actor	varchar(50) NOT NULL
director	varchar(50) NOT NULL
genre	varchar(25) NOT NULL
gross_sales	float
ratings (G, PG, R13, R16,X)	varchar(5)
 - b. Insert at least 5 records
 - c. Create a user named **test_user** and assign a **password** and give it an admin access by checking necessary SQL functions
3. Guided by the Demo code attached in this task. test_DemoDB.py
4. Kindly continue working on the code that will allow the user to navigate through the Database and perform simple CRUD operations. Follow the following **CLI Menu Options**:

```
----- MOVIE DATABASE CLI -----
1. Add Movie
2. View Movies
3. Update Movies
4. Delete a Movie
5. Search a Movie
6. Display Total Records
7. Exit
Select an option (1-6): |
```

5. The user should be able perform the ff: in your program.

MOVIE DATABASE CRUD APP

- 1- Add New Record
 - 2- View all records,
 - 3- Update a Record and show the updates,
 - 4- Delete a record
 - 5- Search A Record**
 - 6- Display **Total Numbers** of Movies stored in the database
 - 7- Exit
6. For additional challenge, Task – You are to add a **SEARCH option** in the MENU that will allow the user to search by Name or emp_id, then display the information about the record being search. You may use Like statement and fetchOne method in my SQL to do this,
 7. You are also going to add a method that will display the the **total number of records** in your database – You may use SQL count statement for this.
 8. What to submit:
 - a. UI Menu
 - b. Sample Output
 - c. Source Code
 - d. Exported sql file

SOURCE CODE:

```
1 import mysql.connector
2
3
4 db = mysql.connector.connect(
5     host="localhost",
6     user="test_user",
7     password="passwordis101010",
8     database="moviesDB"
9 )
10
11 cur = db.cursor()
12
13
14
15 def create_movie():
16     print("\n--- ADD MOVIE RECORD ---")
17     mid = input("Enter Movie ID: ")
18     title = input("Enter Movie Title: ")
19     actor = input("Lead Actor: ")
20     director = input("Director: ")
21     genre = input("Genre: ")
22     sales = float(input("Gross Sales: "))
23     rating = input("Rating (G, PG, R13, R16, X): ")
24
25     query = """
26         INSERT INTO movies (movie_id, title, main_actor, director, genre, gross_sales, ratings)
27         VALUES (%s, %s, %s, %s, %s, %s, %s)
28     """
29
30     cur.execute(query, (mid, title, actor, director, genre, sales, rating))
31     db.commit()
32
33     print("\n✓ Movie successfully added.\n")
34
35
36
37 def show_movies():
38     print("\n--- ALL MOVIE RECORDS ---\n")
39
40     cur.execute("SELECT * FROM movies")
41     records = cur.fetchall()
42
43     if not records:
44         print("No movie records available.")
45         return
46
47     for rec in records:
48         print(f"ID: {rec[0]} | Title: {rec[1]} | Actor: {rec[2]} | Director: {rec[3]} | "
49             f"Genre: {rec[4]} | Sales: {rec[5]} | Rating: {rec[6]}")
50
51     print("\nTotal movies found:", len(records))
```

```

55 def modify_movie():
56     print("\n--- UPDATE MOVIE ---")
57     mid = input("Enter Movie ID to modify: ")
58
59     print("\nEnter updated details:")
60     title = input("New Title: ")
61     actor = input("New Actor: ")
62     director = input("New Director: ")
63     genre = input("New Genre: ")
64     sales = float(input("New Gross Sales: "))
65     rating = input("New Rating: ")
66
67     q = """
68     UPDATE movies
69     SET title=%s, main_actor=%s, director=%s, genre=%s, gross_sales=%s, ratings=%s
70     WHERE movie_id=%s
71     """
72
73     cur.execute(q, (title, actor, director, genre, sales, rating, mid))
74     db.commit()
75
76     print("\n✓ Movie record updated.\n")
77
78
79
80 def remove_movie():
81     print("\n--- DELETE MOVIE ---")
82     mid = input("Enter Movie ID to delete: ")
83
84     cur.execute("DELETE FROM movies WHERE movie_id = %s", (mid,))
85     db.commit()
86
87     print("\n✓ Movie deleted.\n")
88
89
90 def lookup_movie():
91     print("\n--- SEARCH MOVIE ---")
92     key = input("Search keyword (title/actor): ")
93
94     q = "SELECT * FROM movies WHERE title LIKE %s OR main_actor LIKE %s"
95     key_like = f"%{key}%"
96
97     cur.execute(q, (key_like, key_like))
98     result = cur.fetchone()
99
100     if result:
101         print("\nMovie Found:\n")
102         print(f"ID: {result[0]}\nTitle: {result[1]}\nActor: {result[2]}\nDirector: {result[3]}")
103         print(f"Genre: {result[4]}\nGross Sales: {result[5]}\nRating: {result[6]}")
104     else:
105         print("\nNo matching record found.")
106

```

```

108
109 def total_movies():
110     cur.execute("SELECT COUNT(*) FROM movies")
111     total = cur.fetchone()[0]
112     print(f"\nTotal movies stored: {total}\n")
113
114
115
116 def main_menu():
117     while True:
118         print("""
119 =====
120 |      MOVIE RECORD MANAGEMENT      |
121 =====
122 1 • Insert New Movie
123 2 • Display All Movies
124 3 • Edit Movie Details
125 4 • Remove Movie
126 5 • Search Movie
127 6 • Count All Movies
128 7 • Quit Program
129 =====
130 """)
131         option = input("Choose an option: ")
132
133         if option == "1":
134             create_movie()
135         elif option == "2":
136             show_movies()
137         elif option == "3":
138             modify_movie()
139         elif option == "4":
140             remove_movie()
141         elif option == "5":
142             lookup_movie()
143         elif option == "6":
144             total_movies()
145         elif option == "7":
146             print("\nProgram closed.")
147             break
148         else:
149             print("Invalid option. Try again.\n")
150
151
152 |
153 main_menu()

```


Sample Output:

```
=====
| MOVIE DATABASE CRUD APP
=====
1 - Add New Record
2 - View All Records
3 - Update a Record
4 - Delete a Record
5 - Search a Record
6 - Display Total Number of Movies
7 - Exit
=====

Enter choice: 2

=== MOVIE LIST ===
(11, 'Avatar', 'Sam Worthington', 'James Cameron', 'Sci-Fi', 2847.3, 'PG')
(12, 'The Avengers', 'Robert Downey Jr.', 'Joss Whedon', 'Action', 1518.8, 'PG')
(13, 'Jurassic World', 'Chris Pratt', 'Colin Trevorrow', 'Adventure', 1670.5, 'PG')
(14, 'Frozen II', 'Idina Menzel', 'Jennifer Lee', 'Animation', 1450.2, 'G')
(15, 'Black Panther', 'Chadwick Boseman', 'Ryan Coogler', 'Action', 1347.1, 'PG')
(99, 'The Amazing Spider-Man', 'Andrew Garfield', 'Marc Webb', 'Action', 757.9, 'PG')

Total Records: 6

=====
| MOVIE DATABASE CRUD APP
=====
1 - Add New Record
2 - View All Records
3 - Update a Record
4 - Delete a Record
5 - Search a Record
6 - Display Total Number of Movies
7 - Exit
=====

Enter choice: 1

=== ADD NEW MOVIE ===
Movie ID: 120
Title: The Matrix
Main Actor: Keanu Reeves
Director: Lana Wachowski
Genre: Sci-Fi
Gross Sales: 466.2
Rating (G, PG, R13, R16, X): R13

Movie added successfully!
```